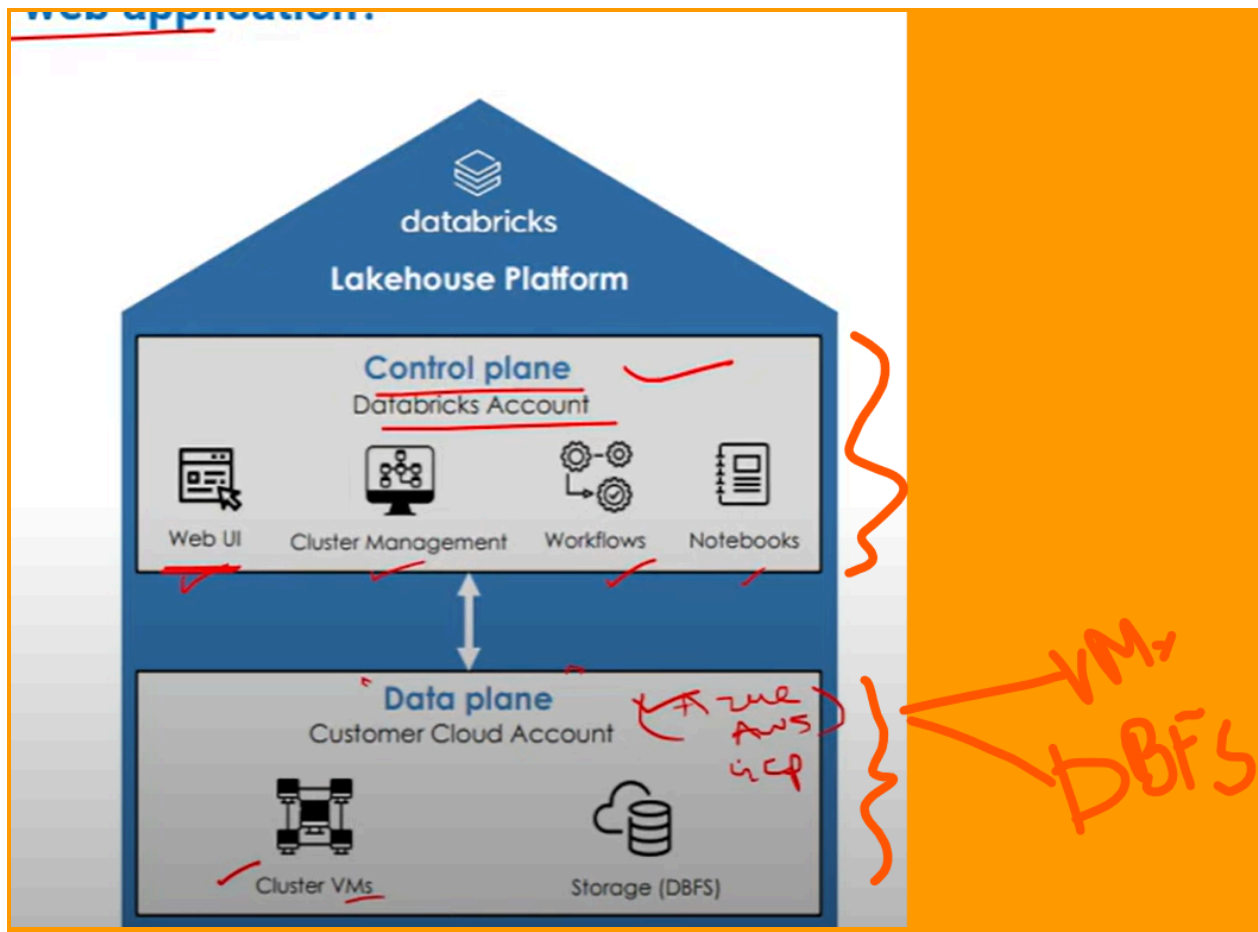




Which of the following benefits of using the Databricks Lakehouse Platform is provided by Delta Lake?

- A. The ability to manipulate the same data using a variety of languages.
- B. The ability to collaborate in real time on a single notebook
- C. The ability to set up alerts for query failures
- D. The ability to support batch and streaming workloads (Answer)
- E. The ability to distribute complex data operations

Question 8:

Which of the following describes the storage organization of a Delta table?

- A. Delta tables are stored in a single file that contains data, history, metadata, and other attributes.
- B. Delta tables store their data in a single file and all metadata in a collection of files in a separate location.
- C. Delta tables are stored in a collection of files that contain data, history, metadata, and other attributes.
- D. Delta tables are stored in a collection of files that contain only the data stored within the table.
- E. Delta tables are stored in a single file that contains only the data stored within the table.

Lakehouse

Collection → delta_log

Optimize:

Question 11:

Which of the following commands can a data engineer use to compact small data files of a Delta table into larger ones?

- A. ~~PARTITION BY~~
- B. ~~ZORDER BY~~
- C. COMPACT
- D. VACUUM
- E. OPTIMIZE (Answer)

create table time travel
① insert
2
3
4
delete id=1
delta log
① Parquet
②
③
④
optimize

VACCUUM:

Question 12:

*select * from table vacuum as 1 2*
A data engineer has realized that they made a mistake when making a daily update to a table. They need to use Delta time travel to restore the table to a version that is 3 days old. However, when the data engineer attempts to time travel to the older version, they are unable to restore the data because the data files have been deleted. *vacuum*

Which of the following explains why the data files are no longer present?

- ☒ A. The VACUUM command was run on the table
- ☐ B. The TIME TRAVEL command was run on the table
- ☐ C. The DELETE HISTORY command was run on the table
- ☐ D. The OPTIMIZE command was run on the table
- ☐ E. The HISTORY command was run on the table

By default, Delta Lake enforces a minimum retention period of 7 days for safety. Running **VACUUM** with **RETAIN 0 HOURS** will fail unless you explicitly disable the retention check using:

```
CopyEditSET
spark.databricks.delta.retentionDurationCheck.enabled =
false;
```

Question 7:

A data analyst has a series of queries in a SQL program. The data analyst wants this program to run every day. They only want the final query in the program to run on Sundays. They ask for help from the data engineering team to complete this task. Which of the following approaches could be used by the data engineering team to complete this task?

- ☒ A. They could submit a feature request with Databricks to add this functionality.
- ☒ B. They could wrap the queries using PySpark and use Python's control flow system to determine when to run the final query. (Answer)
- ☒ C. They could only run the entire program on Sundays.
- ☒ D. They could automatically restrict access to the source table in the final query so that it is only accessible on Sundays.
- ☒ E. They could redesign the data model to separate the data used in the final query into a new table.

Databricks Certified Data Engineer Associate | Exam Preparation- Part 2

Databricks:

Question 15:

A data engineer only wants to execute the final block of a Python program if the Python variable day_of_week is equal to 1 and the Python variable review_period is True. Which of the following control flow statements should the data engineer use to begin this conditionally executed code block?

- A. if day_of_week = 1 and review_period:
- B. if day_of_week = 1 and review_period = "True":
- ☒ C. if day_of_week == 1 and review_period == "True":
- ☒ D. if day_of_week == 1 and review_period:
- E. if day_of_week = 1 & review_period: = "True":

Question 3:

Which of the following describes the relationship between Gold tables and Silver tables?

- ☒ A. Gold tables are more likely to contain aggregations than Silver tables. (Answer)
- ☐ B. Gold tables are more likely to contain valuable data than Silver tables.
- ☒ C. Gold tables are more likely to contain a less refined view of data than Silver tables.
- ☒ D. Gold tables are more likely to contain more data than Silver tables.
- ☒ E. Gold tables are more likely to contain truthful data than Silver tables.



10:51 / 40:46



Naval Yehul

```
.outputMode("append")  
.option("checkpointLocation", "/path")  
.table("Output_Table")
```

Trigger	Method call	Behavior
Unspecified		Default: processingTime="500m"
Fixed interval	<code>.trigger(processingTime="5 minutes")</code>	Process data in micro-batches the user-specified intervals
Triggered batch	<code>.trigger(once=True)</code>	Process all available data in a single batch, then stop
Triggered micro-batches	<code>.trigger(availableNow=True)</code>	Process all available multiple micro-batches

Question 9:

A Delta Live Table pipeline includes two datasets defined using STREAMING LIVE TABLE. Three datasets are defined against Delta Lake table sources using LIVE TABLE. The table is configured to run in Production mode using the Continuous Pipeline Mode. Assuming previously unprocessed data exists and all definitions are valid, what is the expected outcome after clicking Start to update the pipeline?

- A. All datasets will be updated at set intervals until the pipeline is shut down. The compute resources will persist to allow for additional testing.
- B. All datasets will be updated once and the pipeline will persist without any processing. The compute resources will persist but go unused.
- C. All datasets will be updated at set intervals until the pipeline is shut down. The compute resources will be deployed for the update and terminated when the pipeline is stopped. (Answer)
- D. All datasets will be updated once and the pipeline will shut down. The compute resources will be terminated.
- E. All datasets will be updated once and the pipeline will shut down. The compute resources will persist to allow for additional testing.

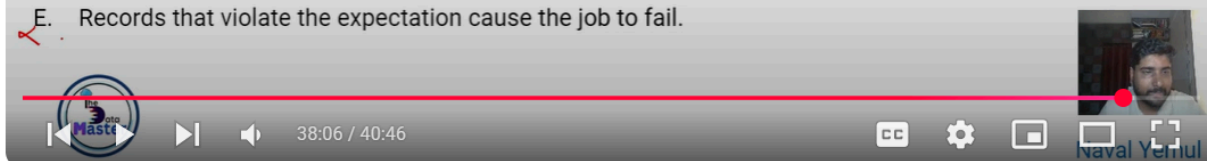


Question 10:

A dataset has been defined using Delta Live Tables and includes an expectations clause: CONSTRAINT valid_timestamp EXPECT (timestamp > '2020-01-01') ON VIOLATION DROP ROW

What is the expected behavior when a batch of data containing data that violates these constraints is processed?

- A. Records that violate the expectation are dropped from the target dataset and loaded into a quarantine table.
- B. Records that violate the expectation are added to the target dataset and flagged as invalid in a field added to the target dataset.
- C. Records that violate the expectation are dropped from the target dataset and recorded as invalid in the event log. (Answer)
- D. Records that violate the expectation are added to the target dataset and recorded as invalid in the event log.
- E. Records that violate the expectation cause the job to fail.



Question 11:

Which of the following describes when to use the CREATE STREAMING LIVE TABLE (formerly CREATE INCREMENTAL LIVE TABLE) syntax over the CREATE LIVE TABLE syntax when creating Delta Live Tables (DLT) tables using SQL?

- A. CREATE STREAMING LIVE TABLE should be used when the subsequent step in the DLT pipeline is static.
- B. CREATE STREAMING LIVE TABLE should be used when data needs to be processed incrementally. (Answer)
- C. CREATE STREAMING LIVE TABLE is redundant for DLT and it does not need to be used.
- D. CREATE STREAMING LIVE TABLE should be used when data needs to be processed through complicated aggregations.
- E. CREATE STREAMING LIVE TABLE should be used when the previous step in the DLT pipeline is static.

What is a Live Table?

Live Tables are materialized views for the lakehouse.

A live table is:

- Defined by a SQL query
- Created and kept up-to-date by a pipeline

```
CREATE OR REFRESH LIVE TABLE report
AS SELECT sum(profit)
FROM prod.sales
```

Live tables provides tools to:

- Manage dependencies
- Control quality
- Automate operations
- Simplify collaboration
- Save costs
- Reduce latency

Clusters:

Clusters

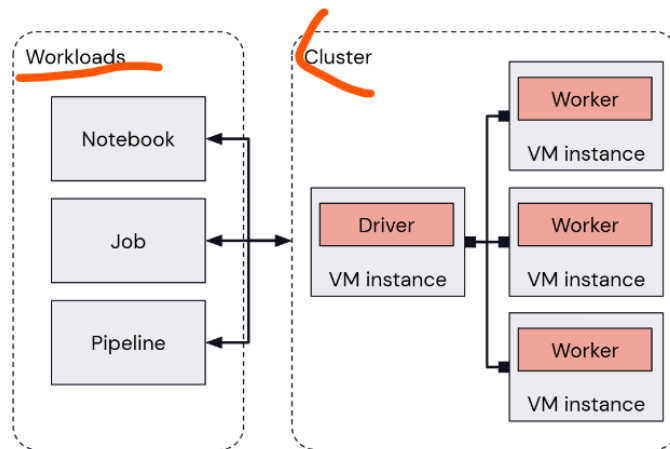
Overview

Collection of VM instances

Distributes workloads across workers

Two main types:

1. **All-purpose** clusters for interactive development
2. **Job** clusters for automating workloads



17

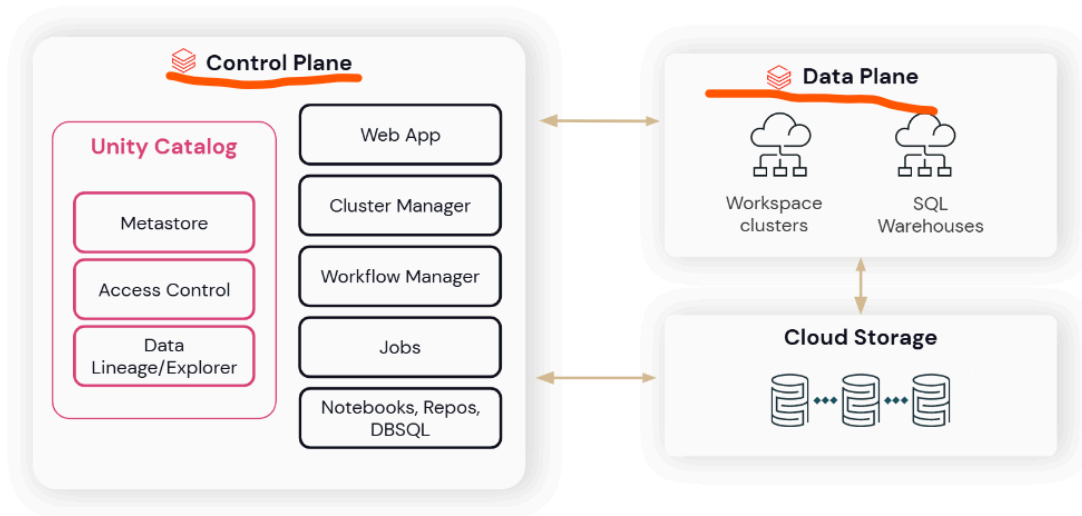


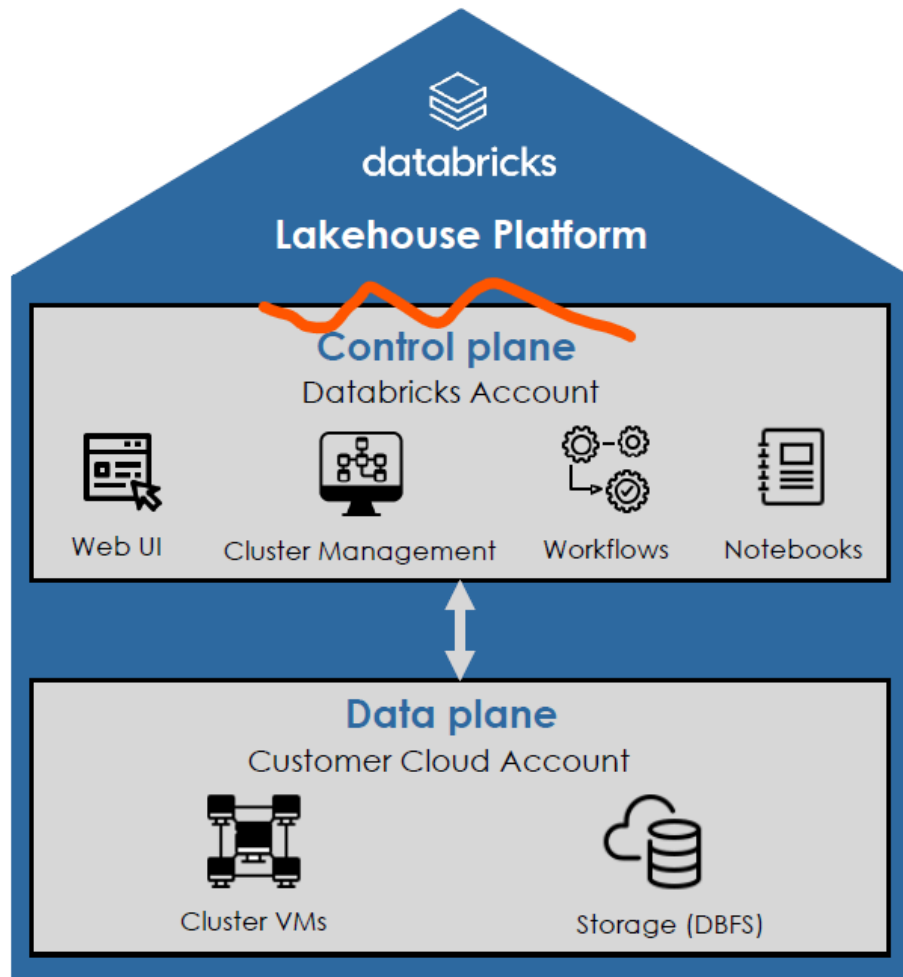
Cluster virtual machines operate in the data plane and are provisioned within the customer's cloud account as part of the Databricks Lakehouse architecture.

In a Spark cluster deployment, both compute and storage resources are managed in the data plane of the lakehouse platform.

Databricks architecture:

Databricks Workspace and Services





Clusters

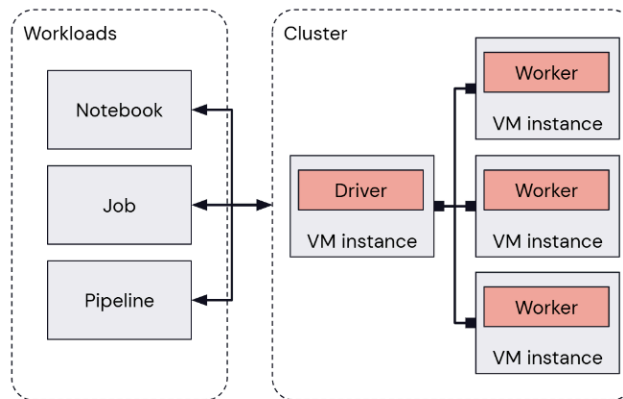
Overview

Collection of VM instances

Distributes workloads across workers

Two main types:

1. **All-purpose** clusters for interactive development
2. **Job** clusters for automating workloads



17



Delta Lake is the default format for tables created in Databricks

```
CREATE TABLE foo
USING DELTA
```

```
df.write
  .format("delta")
```

(Stored) Views	Temp views	Global Temp views
▶ Persisted in DB	▶ Session-scoped	▶ Cluster-scoped
▶ Dropped only by DROP VIEW	▶ dropped when session ends	▶ dropped when cluster restarted
▶ CREATE VIEW	▶ CREATE TEMP VIEW	▶ CREATE GLOBAL TEMP VIEW

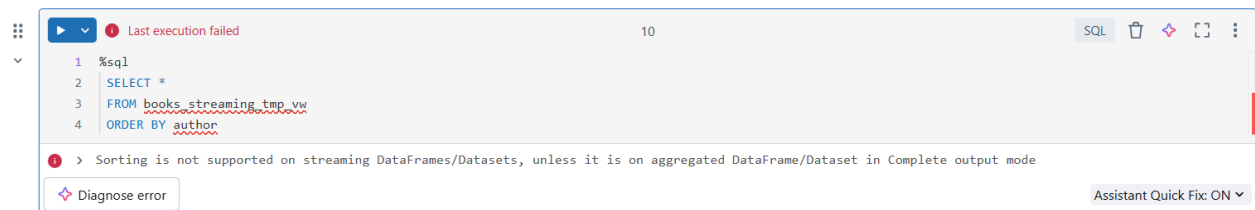
Derar Alhussein © Udemyl | Databricks Certified Data Engineer Associate - Preparation

Checkpointing

```
streamDF.writeStream
  .trigger(processingTime="2 minutes")
  .outputMode("append")
  .option("checkpointLocation", "/path")
  .table("Output_Table")
```

- ▶ Store stream state
- ▶ Track the progress of your stream processing
- ▶ Can **Not** be shared between separate streams

```
(spark.readStream  
  .table("books")  
  .createOrReplaceTempView("books_streaming_tmp_vw")  
)
```



Delta Lake vs Delta Live:

Incremental Data Ingestion

- ▶ Loading new data files encountered since the last ingestion
- ▶ Reduces redundant processing
- ▶ 2 mechanisms:
 - ▶ COPY INTO
 - ▶ Auto loader

```
spark.readStream .format("cloudFiles")  
  .option("cloudFiles.format", )  
  .load('/path/to/files')  
  .writeStream  
  .option("checkpointLocation", )  
  .table()
```


spark.readStream

.format("cloudFiles")

.option("cloudFiles.format", <source_format>)

.option("cloudFiles.schemaLocation", <schema_directory>)

.load('/path/to/files')

.writeStream

→ .option("checkpointLocation", <checkpoint_directory>)

→ .option("mergeSchema", "true")

.table(<table_name>)

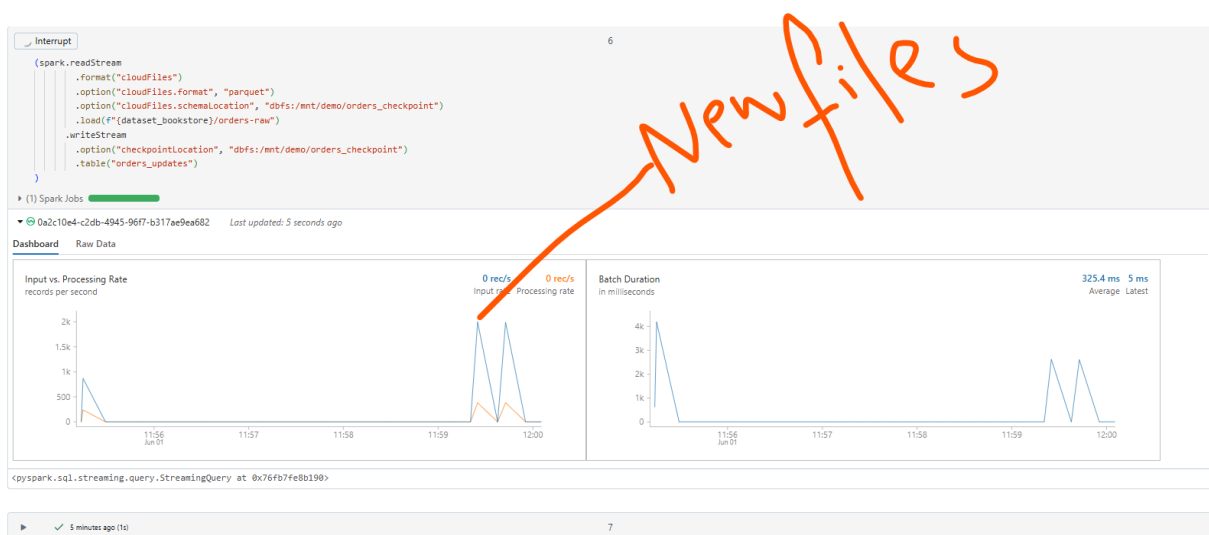
Derar Alhussein © Udemry | Databricks Certified Data Engineer Associate - Preparation

format--cloudfiles

Option, merge schema,checkpointing

Clodfiles.format and

cloudfiles,.schemalocaliotn



By default, if you don't provide any trigger interval, the data will be processed every half second. This is equivalent to

```
trigger(processingTime="500ms")
```

Databricks Jobs allow to orchestrate data processing tasks. This means the ability to run and manage multiple tasks as a directed acyclic graph (DAG) in a job.

A data engineer has a Job with multiple tasks that takes more than 2 hours to complete. In the last run, the final task unexpectedly failed.

They can repair this Job Run so only the failed tasks will be re-executed

Databricks Jobs allow to orchestrate data processing tasks. This means the ability to run and manage multiple tasks as a directed acyclic graph (DAG) in a job.

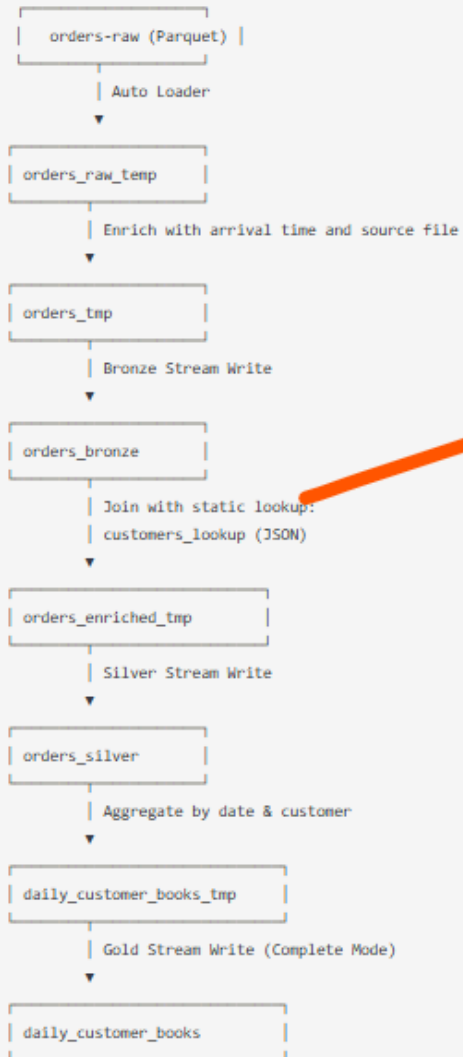
Multihop:

```
(spark.readStream
```

```
.format("cloudFiles")
.option("cloudFiles.format", "parquet")
.option("cloudFiles.schemaLocation",
"dbfs:/mnt/demo/checkpoints/orders_raw")
.load(f"{dataset_bookstore}/orders-raw")
.createOrReplaceTempView("orders_raw_temp"))
```

```
%sql
CREATE OR REPLACE TEMPORARY VIEW orders_tmp AS (
  SELECT *, current_timestamp() arrival_time, input_file_name() source_file
  FROM orders_raw_temp
)
```

plaintext



Static
lookup

DLT

```
CREATE OR REFRESH STREAMING LIVE TABLE orders_cleaned (  
  CONSTRAINT valid_order_number EXPECT (order_id IS NOT NULL) ON VIOLATION DROP ROW  
)  
COMMENT "The cleaned books orders with valid order_id"  
AS  
  SELECT order_id, quantity, o.customer_id, c.profile:first_name as f_name, c.profile:last_name as l_name,  
         cast(from_unixtime(order_timestamp, 'yyyy-MM-dd HH:mm:ss') AS timestamp) order_timestamp, o.book_id,  
         c.profile:address:country as country  
  FROM STREAM(LIVE.orders_raw) o  
  LEFT JOIN LIVE.customers c  
    ON o.customer_id = c.customer_id
```

Overall explanation

- **Delta Live Tables (DLT)** can run in **two modes: Continuous Mode** and **Triggered Mode**.
 - **Continuous Mode** means that the pipeline continuously processes new data as it arrives, which aligns with the behavior of keeping the compute resources active for rapid testing.
 - **Triggered Mode** involves batch processing at specified intervals rather than continuously running.
- **Development settings** typically involve running pipelines in continuous mode for rapid testing and debugging, allowing for the constant ingestion and processing of data. These settings do not require long-running production-like stability, which fits the description provided.
- **Production settings** generally focus on stability, so they would more likely use **Triggered Mode**, where the pipeline is triggered at fixed intervals, allowing for more controlled operations.

Given that the pipeline is designed for rapid testing and remains active (which suggests continuous operation), and considering that it hasn't been explicitly stated as a production environment, **Development settings** are the most appropriate.

Development--continuous mode.

Production-- triggered mode

CDC:DLT live tables ,run pipeline

The screenshot displays the Databricks workspace interface for a pipeline named 'demobookstore'. The top navigation bar includes the Databricks logo, a search bar, and a 'CTRL + P' shortcut. Below the navigation bar, the pipeline is shown in 'Development' mode. The 'Run' status is 'Completed' at 6/2/2025, 8:52:15 AM. The pipeline graph shows a flow from 'orders_raw' (Streaming table) to 'orders_cleaned' (Streaming table), which then branches into 'cn_daily_customer...' (Materialized view) and 'fr_daily_ci...' (Materialized view). The 'customers' table (Materialized view) also feeds into 'orders_cleaned'. The 'Pipeline details' panel on the right shows the Pipeline ID, type (ETL pipeline), source code, and run as 'arya ss'. The 'Event log' at the bottom shows a sequence of events: 'user_action' (User mithelesharya@gmail.com started an update), 'create_update' (Update 1e4951 started by API_CALL), 'update_progress' (Update 1e4951 is WAITING_FOR_RESOURCES), and 'update_progress' (Update 1e4951 is INITIALIZING).

Pipeline details

Property	Value
Pipeline ID	529817eb-cc79-48ce-a37c-db38fce5e
Pipeline type	ETL pipeline
Source code	/Users/mithelesharya@gmail.com/Co... Production Pipelines/4.1 - Delta Live T
Run as	arya ss
Tags	None

Event log

Time	Event Type	Message
2 hours ago	user_action	User mithelesharya@gmail.com started an update.
2 hours ago	create_update	Update 1e4951 started by API_CALL.
2 hours ago	update_progress	Update 1e4951 is WAITING_FOR_RESOURCES.
2 hours ago	update_progress	Update 1e4951 is INITIALIZING.

```
CREATE LIVE TABLE aggregated_sales
AS
  SELECT store_id, sum(total)
  FROM LIVE.cleaned_sales
  GROUP BY store_id
```

results:

10:59 AM (1s)					1
<pre>files = dbutils.fs.ls("dbfs:/mnt/demo-datasets/bookstore") display(files)</pre>					
(2) Spark Jobs					
Table +					
	path	name	size	modificationTime	
1	dbfs:/mnt/demo-datasets/bookstore/autoloader/	autoloader/	0	1748869352000	
2	dbfs:/mnt/demo-datasets/bookstore/books-cdc/	books-cdc/	0	1748390655000	
3	dbfs:/mnt/demo-datasets/bookstore/books-csv/	books-csv/	0	1748390659000	
4	dbfs:/mnt/demo-datasets/bookstore/books-csv-new/	books-csv-new/	0	1748390657000	
5	dbfs:/mnt/demo-datasets/bookstore/books-streaming/	books-streaming/	0	1748390662000	
6	dbfs:/mnt/demo-datasets/bookstore/checkpoints/	checkpoints/	0	1748869374000	

Just now (< 1s)					2	Python			
<pre>files = dbutils.fs.ls("dbfs:/mnt/demo-datasets/bookstore/system/events") display(files)</pre>									
(2) Spark Jobs									
Table +									
	path	name	size	mod					
1	dbfs:/mnt/demo-datasets/bookstore/system/events/_delta_log/	_delta_log/	0						
2	dbfs:/mnt/demo-datasets/bookstore/system/events/part-00000-019887cb-de7c-4211-a401-40f43b9ead34-c000.snappy.parquet	part-00000-019887cb-de7c-4211-a401-40f43b9ead34-c000.snappy.parquet	11961						
3	dbfs:/mnt/demo-datasets/bookstore/system/events/part-00000-0200d2dc-9857-462a-9236-cc56389fa3e7-c000.snappy.parquet	part-00000-0200d2dc-9857-462a-9236-cc56389fa3e7-c000.snappy.parquet	11962						
4	dbfs:/mnt/demo-datasets/bookstore/system/events/part-00000-040a764f-b593-4e1d-9222-1d521009aecc-c000.snappy.parquet	part-00000-040a764f-b593-4e1d-9222-1d521009aecc-c000.snappy.parquet	11962						
5	dbfs:/mnt/demo-datasets/bookstore/system/events/part-00000-0459ce04-1540-42a9-bbcd-a11b1d0c15e4-c000.snappy.parquet	part-00000-0459ce04-1540-42a9-bbcd-a11b1d0c15e4-c000.snappy.parquet	11962						
6	dbfs:/mnt/demo-datasets/bookstore/system/events/part-00000-07d0c52a-0eee-4cc0-82ea-e01729e016c-c000.snappy.parquet	part-00000-07d0c52a-0eee-4cc0-82ea-e01729e016c-c000.snappy.parquet	11962						
7	dbfs:/mnt/demo-datasets/bookstore/system/events/part-00000-087dfba5-2e6e-4acd-9098-1b267ccf2ea8-c000.snappy.parquet	part-00000-087dfba5-2e6e-4acd-9098-1b267ccf2ea8-c000.snappy.parquet	11962						
8	dbfs:/mnt/demo-datasets/bookstore/system/events/part-00000-0d074865-b766-4c13-8ca4-ca68106bba72-c000.snappy.parquet	part-00000-0d074865-b766-4c13-8ca4-ca68106bba72-c000.snappy.parquet	11961						
9	dbfs:/mnt/demo-datasets/bookstore/system/events/part-00000-0e30c2d1-7bd9-48f5-88fe-e8e8f8a4b854-c000.snappy.parquet	part-00000-0e30c2d1-7bd9-48f5-88fe-e8e8f8a4b854-c000.snappy.parquet	11962						
10	dbfs:/mnt/demo-datasets/bookstore/system/events/part-00000-0f368665-c7c9-422d-b89c-d2e08524a1de-c000.snappy.parquet	part-00000-0f368665-c7c9-422d-b89c-d2e08524a1de-c000.snappy.parquet	11962						
11	dbfs:/mnt/demo-datasets/bookstore/system/events/part-00000-0f888391-ce97-4d04-a1bc-0b02980a52ae-c000.snappy.parquet	part-00000-0f888391-ce97-4d04-a1bc-0b02980a52ae-c000.snappy.parquet	11962						
12	dbfs:/mnt/demo-datasets/bookstore/system/events/part-00000-107c1707-ef49-449e-81cb-86da23afbabc-c000.snappy.parquet	part-00000-107c1707-ef49-449e-81cb-86da23afbabc-c000.snappy.parquet	11962						
13	dbfs:/mnt/demo-datasets/bookstore/system/events/part-00000-11c1a80a-5786-41da-9bc6-ffea9c808967-c000.snappy.parquet	part-00000-11c1a80a-5786-41da-9bc6-ffea9c808967-c000.snappy.parquet	11962						

Schema grants:

New Query 2025-06-02 10:04pm

New Query 2025-06-02 10:16pm

+

▶ Run selected (1000)

hive_metastore.default

New SQL editor: OFF

```
1 GRANT Select ,Modify, read_metadata , create on schema hr_db to hr_team;
2
3
4 GRANT USAGE ON SCHEMA hr_db TO hr_team;
5
6 GRANT select ON VIEW hr_db.paris_emp TO `sma@company.com`;
7
8 show grants on schema hr_db;
```

Raw results

+

	^A _C Principal	^A _C ActionType	^A _C ObjectType	^A _C ObjectKey
1	hr_team	SELECT	DATABASE	hr_db
2	hr_team	CREATE	DATABASE	hr_db
3	hr_team	MODIFY	DATABASE	hr_db
4	hr_team	READ_METADATA	DATABASE	hr_db
5	hr_team	USAGE	DATABASE	hr_db
6	mithelesharya@gmail.co...	OWN	DATABASE	hr_db

🕒 187 ms | 6 rows returned



You need to add email to users to grant access to specific users

```
GRANT EXECUTE ON FUNCTION prod.ml_team.iris_model TO `ml-team-acme`;
```

```
GRANT MODIFY ON TABLE transactions TO  
alex.johnson@productanalytics.com
```

```
-- Transferring ownership of an object to `some-group`  
> ALTER SCHEMA some_schema OWNER TO `some-group`;
```

Account groups:

Account groups can be created by account admins and workspace admins o

Workspace-local groups

can be created only by workspace admins.

Autoloader concept:

Auto loader monitors a source location, in which files accumulate, to identify and ingest only new arriving files with each command run. While the files that have already been ingested in previous runs are skipped.

Checkpointing is the mechanism used by Auto Loader to maintain ingestion progress and discovered file metadata, ensuring reliable and incremental data processing.

Auto Loader

```
▶ 23 hours ago 6

(spark.readStream
  .format("cloudFiles")
  .option("cloudFiles.format", "parquet")
  .option("cloudFiles.schemaLocation", "dbfs:/mnt/demo/orders_checkpoint")
  .load(f"{dataset_bookstore}/orders-raw")
  .writeStream
    .option("checkpointLocation", "dbfs:/mnt/demo/orders_checkpoint")
    .table("orders_updates")
)

▶ (1) Spark Jobs

▶ 0a2c10e4-c2db-4945-96f7-b317ae9ea682 Last updated: 23 hours ago

<pyspark.sql.streaming.query.StreamingQuery at 0x76fb7fe8b190>
```

Action	SQL syntax	Python syntax	Result
warn (default)	EXPECT	dlt.expect	Invalid records are written to the target.
drop	EXPECT ... ON VIOLATION DROP ROW	dlt.expect_or_drop	Invalid records are dropped before data is written to the target. The count of dropped records is logged alongside other dataset metrics.
fail	EXPECT ... ON VIOLATION FAIL UPDATE	dlt.expect_or_fail	Invalid records prevent the update from succeeding. Manual intervention is required before reprocessing. This expectation causes a failure of a single flow and does not cause other flows in your pipeline to fail.

Modi
Build

Lear
Table

📖 |

Mor

Lear
DLT
histc

Tuto

`pipelines.reset.allowed = false` table,C) To prevent users from performing a full refresh on the table.

B) `skipChangeCommits = true`, you want your downstream DLT streaming table to ignore these changes and only process appends.

Production pipelines:

The data engineer team has a DLT pipeline that updates all the tables at defined intervals until manually stopped. The

compute resources terminate when the pipeline is stopped.

The DLT pipeline executes in Continuous Pipeline mode under Production mode.

In DLT, what is the best practice for ensuring the idempotency of writes to multiple target tables within a `foreachBatch` pattern? A) Using `MERGE INTO` statement directly. B) Relying on DLT's internal transaction management. C) Providing `txnAppId` and `txnVersion` options to the `DataFrameWriter`. D) Manually managing commit offsets.

What is the impact of setting `pipelines.clusterShutdown.delay` in DLT development mode? A) It defines how long the cluster waits before scaling up. B) It specifies how long the cluster remains active after the last update in development mode to avoid frequent restarts.

ViewS:

- **TEMPORARY**

TEMPORARY views are visible only to the session that created them and are dropped when the session ends.

- **GLOBAL TEMPORARY**

Applies to:  Databricks Runtime

GLOBAL TEMPORARY views are tied to a system preserved temporary schema `global_temp`.

Dynamic views in Databricks allow fine-grained access control by returning different results based on user group membership. These are implemented using conditional SQL logic with functions like ``is_member()``.

Pipelines:

Triggered pipelines update each table with whatever data is currently available and then they shut down.

In Production mode, the Delta Live Tables system:

- Terminates the cluster immediately when the pipeline is stopped.
- Restarts the cluster for recoverable errors (e.g., memory leak or stale credentials).
- Retries execution in case of specific errors (e.g., a failure to start a cluster)

GIT:

One advantage of Databricks Repos over the built-in Databricks Notebooks versioning is that Databricks Repos supports creating and managing branches for development work.

- Clone, push to, and pull from a remote Git repository.
- Create and manage branches for development work, including merging, rebasing, and resolving conflicts.
- Create notebooks (including IPYNB notebooks) and edit them and other files.
- Visually compare differences upon commit and resolve merge conflicts.

This is correct. Databricks Repos supports Git operations including commit, comment, and push to remote branches.

MERGE:

Used for conditional updates and inserts (upserts), not for full table replacement.

Which of the following code blocks will apply a 50% discount on all the products where the price is greater than 1000 and save the new price to the table?

The UPDATE statement is used to modify the existing records in a table that match the WHERE condition. In this case, we are updating the products where the price is strictly greater than 1000.

Syntax:

```
UPDATE table_name
SET column_name = expr
WHERE condition
```

1. **INSERT OVERWRITE** replaces all existing records in a Delta table with new data in a single, efficient operation.
2. It is ideal for full refresh scenarios, like batch jobs that generate a complete dataset every time.

Constraint:

```
ALTER TABLE people10m ADD CONSTRAINT validIds CHECK (id > 1 and id < 99999999) ;
```

A data engineer has defined the following data quality constraint in a Delta Live Tables pipeline:

```
CONSTRAINT valid_id EXPECT (id IS NOT NULL) _____
```

Fill in the above blank so records violating this constraint will be added to the target table, and reported in metrics

☐ ON VIOLATION ADD ROW

☐ ON VIOLATION FAIL UPDATE

☐ ON VIOLATION SUCCESS UPDATE

Your answer is incorrect

☒ ON VIOLATION NULL

Correct answer

☐ There is no need to add ON VIOLATION clause. By default, records violating the constraint will be kept, and reported as invalid in the event log

Concept of cluster pools:

Question #: 94

A data engineer needs to determine whether to use the built-in Databricks Notebooks versioning or version their project using Databricks Repos.

What is an advantage of using Databricks Repos over the Databricks Notebooks versioning?

- A. Databricks Repos allows users to revert to previous versions of a notebook
- B. Databricks Repos is wholly housed within the Databricks Data Intelligence Platform
- C. Databricks Repos provides the ability to comment on specific changes
- D. Databricks Repos supports the use of multiple branches

d

Question #: 105

A data analyst has created a Delta table sales that is used by the entire data analysis team. They want help from the data engineering team to implement a series of tests to ensure the data is clean. However, the data engineering team uses Python for its tests rather than SQL.

Which command could the data engineering team use to access sales in PySpark?

- A. `SELECT * FROM sales`
- B. `spark.table("sales")`
- C. `spark.sql("sales")`
- D. `spark.delta.table("sales")`

B

Question #: 109

What is a benefit of creating an external table from Parquet rather than CSV when using a `CREATE TABLE AS SELECT` statement?

- A. Parquet files can be partitioned
- B. Parquet files will become Delta tables
- C. Parquet files have a well-defined schema
- D. Parquet files have the ability to be optimized

Long format to wide format using ,CONVERT

Question #: 111

A data engineer has a Python variable `table_name` that they would like to use in a SQL query. They want to construct a Python code block that will run the query using `table_name`.

They have the following incomplete code block:

```
_____(f"SELECT customer_id, spend FROM {table_name}")
```

What can be used to fill in the blank to successfully complete the task?

- A. `spark.delta.sql`
- B. `spark.sql` 
- C. `spark.table`
- D. `dbutils.sql`

Question #: 112

A data engineer is working with two tables. Each of these tables is displayed below in its entirety.

```
sales
customer_id spend units
a1          28.94  7
a3          874.1223
a4           8.99  1
```

```
favorite_stores
```

```
customer_id store_id
a1           s1
a2           s1
a4           s2
```

Question #: 115

A data engineer has configured a Structured Streaming job to read from a table, manipulate the data, and then perform a streaming write into a new table.

The code block used by the data engineer is below:

```
(spark.table("sales")
  .withColumn("avg_price", col("sales") / col("units"))
  .writeStream
  .option("checkpointLocation", checkpointPath)
  .outputMode("complete")
  .trigger(
    .table("new_sales")
  )
```

Which line of code should the data engineer use to fill in the blank if the data engineer only wants the query to execute a micro-batch to process data every 5 seconds?

- A. trigger("5 seconds")
- B. trigger(continuous="5 seconds")
- C. trigger(once="5 seconds")
- D. trigger(processingTime="5 seconds")

Question #: 117

A data engineer has three tables in a Delta Live Tables (DLT) pipeline. They have configured the pipeline to drop invalid records at each table. They notice that some data is being dropped due to quality concerns at some point in the DLT pipeline. They would like to determine at which table in their pipeline the data is being dropped.

Which approach can the data engineer take to identify the table that is dropping the records?

- A. They can set up separate expectations for each table when developing their DLT pipeline.
- B. They can navigate to the DLT pipeline page, click on the "Error" button, and review the present errors.
- C. They can set up DLT to notify them via email when records are dropped.
- D. They can navigate to the DLT pipeline page, click on each table, and view the data quality statistics.

d

Question #: 120

What describes when to use the CREATE STREAMING LIVE TABLE (formerly CREATE INCREMENTAL LIVE TABLE) syntax over the CREATE LIVE TABLE syntax when creating Delta Live Tables (DLT) tables using SQL?

- A. CREATE STREAMING LIVE TABLE should be used when the subsequent step in the DLT pipeline is static.
- B. CREATE STREAMING LIVE TABLE should be used when data needs to be processed incrementally.
- C. CREATE STREAMING LIVE TABLE should be used when data needs to be processed through complicated aggregations.
- D. CREATE STREAMING LIVE TABLE should be used when the previous step in the DLT pipeline is static.

b

Question #: 121

A Delta Live Table pipeline includes two datasets defined using STREAMING LIVE TABLE. Three datasets are defined against Delta Lake table sources using LIVE TABLE.

The table is configured to run in Production mode using the Continuous Pipeline Mode.

What is the expected outcome after clicking Start to update the pipeline assuming previously unprocessed data exists and all definitions are valid?

- A. All datasets will be updated at set intervals until the pipeline is shut down. The compute resources will persist to allow for additional testing.
- B. All datasets will be updated once and the pipeline will shut down. The compute resources will persist to allow for additional testing.
- C. All datasets will be updated at set intervals until the pipeline is shut down. The compute resources will be deployed for the update and terminated when the pipeline is stopped.
- D. All datasets will be updated once and the pipeline will shut down. The compute resources will be terminated.

C

Question #: 122

A data engineer has developed a data pipeline to ingest data from a JSON source using Auto Loader, but the engineer has not provided any type inference or schema hints in their pipeline. Upon reviewing the data, the data engineer has noticed that all of the columns in the target table are of the string type despite some of the fields only including float or boolean values.

Why has Auto Loader inferred all of the columns to be of the string type?

- A. Auto Loader cannot infer the schema of ingested data
- B. JSON data is a text-based format
- C. Auto Loader only works with string data
- D. All of the fields had at least one null value

b

Question #: 126

A dataset has been defined using Delta Live Tables and includes an expectations clause:

```
CONSTRAINT valid_timestamp EXPECT (timestamp > '2020-01-01') ON VIOLATION DROP ROW
```

What is the expected behavior when a batch of data containing data that violates these constraints is processed?

- A. Records that violate the expectation cause the job to fail.
- B. Records that violate the expectation are added to the target dataset and flagged as invalid in a field added to the target dataset.
- C. Records that violate the expectation are dropped from the target dataset and recorded as invalid in the event log.
- D. Records that violate the expectation are added to the target dataset and recorded as invalid in the event log.

Question #: 129

A single Job runs two notebooks as two separate tasks. A data engineer has noticed that one of the notebooks is running slowly in the Job's current run. The data engineer asks a tech lead for help in identifying why this might be the case.

Which approach can the tech lead use to identify why the notebook is running slowly as part of the Job?

- A. They can navigate to the Runs tab in the Jobs UI to immediately review the processing notebook.
- B. They can navigate to the Tasks tab in the Jobs UI and click on the active run to review the processing notebook.
- C. They can navigate to the Runs tab in the Jobs UI and click on the active run to review the processing notebook.
- D. They can navigate to the Tasks tab in the Jobs UI to immediately review the processing notebook.

C

Question #: 130

A data analysis team has noticed that their Databricks SQL queries are running too slowly when connected to their always-on SQL endpoint. They claim that this issue is present when many members of the team are running small queries simultaneously. They ask the data engineering team for help. The data engineering team notices that each of the team's queries uses the same SQL endpoint.

Which approach can the data engineering team use to improve the latency of the team's queries?

- A. They can increase the cluster size of the SQL endpoint.
- B. They can increase the maximum bound of the SQL endpoint's scaling range.
- C. They can turn on the Auto Stop feature for the SQL endpoint.
- D. They can turn on the Serverless feature for the SQL endpoint.

Question #: 136

Differentiate between all-purpose clusters and jobs clusters.

A data engineering team has created a python notebook to load data from cloud storage, this job has been tested and now needs to be scheduled in production.

Which would be the best cluster to be used in this case?

- A. All purpose cluster
- B. Any Unity Catalog-enabled cluster
- C. Jobs Cluster
- D. Serverless SQL warehouse

Question #: 138

Which two components function in the DB platform architecture's control plane? (Choose two.)

- A. Virtual Machines
- B. Compute Orchestration
- C. Serverless Compute
- D. Compute
- E. Unity Catalog

Question #: 141

A data engineer has configured a Structured Streaming job to read from a table, manipulate the data, and then perform a streaming write into a new table.

The code block used by the data engineer is below:

```
(spark.readStream
  .table("sales")
  .withColumn("avg_price", col("sales") / col("units"))
  .writeStream
  .option("checkpointLocation", checkpointPath)
  .outputMode("complete")
  .          
  .table("new_sales")
)
```

The data engineer only wants the query to process all of the available data in as many batches as required.

Which line of code should the data engineer use to fill in the blank?

- A. trigger(availableNow=True)
- B. trigger(processingTime= "once")
- C. trigger(continuous= "once")
- D. trigger(once=True)

A

Question #: 147

Which method should a Data Engineer apply to ensure Workflows are being triggered on schedule?

- A. Scheduled Workflows require an always-running cluster, which is more expensive but reduces processing latency.
- B. Scheduled Workflows process data as it arrives at configured sources.
- C. Scheduled Workflows can reduce resource consumption and expense since the cluster runs only long enough to execute the pipeline.
- D. Scheduled Workflows run continuously until manually stopped.

C

Question #: 149

An engineering manager uses a Databricks SQL query to monitor ingestion latency for each data source. The manager checks the results of the query every day, but they are manually rerunning the query each day and waiting for the results.

Which of the following approaches can the manager use to ensure the results of the query are updated each day?

- A. They can schedule the query to refresh every 1 day from the SQL endpoint's page in Databricks SQL.
- B. They can schedule the query to refresh every 12 hours from the SQL endpoint's page in Databricks SQL.
- C. They can schedule the query to refresh every 1 day from the query's page in Databricks SQL.
- D. They can schedule the query to run every 12 hours from the Jobs UI.

C

Mm

Question #: 157

How can Git operations must be performed outside of Databricks Repos?

- A. Commit
- B. Pull
- C. Merge
- D. Clone

Question #: 160

A data organization leader is upset about the data analysis team's reports being different from the data engineering team's reports. The leader believes the siloed nature of their organization's data engineering and data analysis architectures is to blame.

Which of the following describes how a data lakehouse could alleviate this issue?

- A. Both teams would respond more quickly to ad-hoc requests
- B. Both teams would use the same source of truth for their work
- C. Both teams would reorganize to report to the same department
- D. Both teams would be able to collaborate on projects in real-time

B

Question #: 163

A data engineer and data analyst are working together on a data pipeline. The data engineer is working on the raw, bronze, and silver layers of the pipeline using Python, and the data analyst is working on the gold layer of the pipeline using SQL. The raw source of the pipeline is a streaming input. They now want to migrate their pipeline to use Delta Live Tables.

Which of the following changes will need to be made to the pipeline when migrating to Delta Live Tables?

- A. The pipeline will need to be written entirely in Python
- B. The pipeline will need to stop using the medallion-based multi-hop architecture
- C. The pipeline will need to be written entirely in SQL
- D. The pipeline can have different notebook sources in SQL & Python.

d

Question #: 168

What is stored in the Databricks customer's cloud account?

- A. Databricks web application
- B. Cluster management metadata
- C. Notebooks
- D. Data

D

Question #: 170

Which of the following commands will return the number of null values in the member_id column?

- A. SELECT count(member_id) FROM my_table;
- B. SELECT count(member_id) - count_null(member_id) FROM my_table;
- C. SELECT count_if(member_id IS NULL) FROM my_table;
- D. SELECT null(member_id) FROM my_table;

Question #: 139

In a healthcare provider organization using Delta Lake to store electronic health records (EHRs), a data analyst needs to analyze a snapshot of the patient_records table from two weeks ago before some recent data corrections were applied.

What approach should the Data Engineer take to allow the analyst to query that specific prior version?

- A. Truncate the table to remove all data, then reload the data from two weeks ago into the truncated table for the analyst to query.
- B. Identify the version number corresponding to two weeks ago from the Delta transaction log, share that version number with the analyst to query using VERSION AS OF syntax, or export that version to a new Delta table for the analyst to query.
- C. Restore the table to the version from two weeks ago using the RESTORE command, and have the analyst query the restored table.
- D. Use the VACUUM command to remove all versions of the table older than two weeks, then the analyst can query the remaining version.

B

RESTORE --- restores for everyone, globally.

SQL END point controls--computer resources not query level

Autoloader---incremental ingestion ,
tracking using metadata checkpoints
Only new files are ingested

Question #: 141

A data engineer has configured a Structured Streaming job to read from a table, manipulate the data, and then perform a streaming write into a new table.

The code block used by the data engineer is below:

```
(spark.readStream
  .table("sales")
  .withColumn("avg_price", col("sales") / col("units"))
  .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("complete")
    .table("new_sales")
)
```

The data engineer only wants the query to process all of the available data in as many batches as required.

Which line of code should the data engineer use to fill in the blank?

- A. trigger(availableNow=True)
- B. trigger(processingTime= "once")
- C. trigger(continuous= "once")
- D. trigger(once=True)

Databricks Certified Data Engineer Associate

Continuous

Overall explanation

- **Delta Live Tables (DLT)** can run in **two modes: Continuous Mode** and **Triggered Mode**.
 - **Continuous Mode** means that the pipeline continuously processes new data as it arrives, which aligns with the behavior of keeping the compute resources active for rapid testing.
 - **Triggered Mode** involves batch processing at specified intervals rather than continuously running.
- **Development settings** typically involve running pipelines in continuous mode for rapid testing and debugging, allowing for the constant ingestion and processing of data. These settings do not require long-running production-like stability, which fits the description provided.
- **Production settings** generally focus on stability, so they would more likely use **Triggered Mode**, where the pipeline is triggered at fixed intervals, allowing for more controlled operations.

Given that the pipeline is designed for rapid testing and remains active (which suggests continuous operation), and considering that it hasn't been explicitly stated as a production environment, **Development settings** are the most appropriate.

`trigger(availableNow=True)` or `Trigger.AvailableNow()` (Process all available data in multiple batches):

If no `trigger()` option is specified, Structured Streaming defaults to a fixed-interval micro-batch of 500ms.

`trigger(availableNow=True)` or `Trigger.AvailableNow()` (Process all available data in multiple batches):

- - `g Trigger.AvailableNow` for all incremental batch processing workloads.

`trigger(availableNow=True)` or `Trigger.AvailableNow()` (Process all available data in multiple batches):

-

Auto Stop determines whether the warehouse stops if it's idle for the specified number of minutes. Idle SQL warehouses continue to accumulate DBU and cloud instance charges until they are stopped.

Correct. Auto Stop automatically shuts down the compute resources when they are not in use, reducing costs.

Auto Loader to incrementally and efficiently processes new data files as they arrive in cloud storage.

Auto Loader is used for incremental data ingestion but does not manage complex data dependencies.

Workspace admins and users with CAN MANAGE privileges on a SQL warehouse can complete the following tasks on an existing SQL warehouse:

Streaming tables:

Streaming tables are designed for append-only data sources and process inputs only once.

```
CREATE OR REFRESH STREAMING TABLE customers_bronze AS SELECT * FROM STREAM
read_files( "/volumes/path/to/files",

format => "json" );
```

Streaming tables are a good choice for data ingestion for the following reasons:

- Each input row is handled only once, which models the vast majority of ingestion workloads (that is, by appending or upserting rows into a table).
- They can handle large volumes of append-only data.

Materialized views are always correct because they automatically recompute joins when dimensions change

Summary Table:

Syntax	Streaming Read?	Context	Source Type	Explicit?	
<code>FROM</code> <code>STREAM(samples.nyctaxi.trips)</code>	✓ Yes	General streaming SQL	Table (static or streaming)	✓ Explicit	”
<code>FROM</code> <code>STREAM(LIVE.taxi_raw_records)</code>	✓ Yes	Delta Live Tables	Managed stream input	✓ Explicit	
<code>FROM</code> <code>live.taxi_raw_records</code>	✗ Maybe (depends)	Delta Live Tables	Managed input	✗ Not explicit	

✓ Recommendation:

- Use `FROM STREAM(...)` when you **need real-time ingestion**.
- Use `FROM live...` without `STREAM()` only if you're **certain** you're doing batch-style transformations or relying on **pipeline configuration** to infer streaming.

Others:

se Databricks secrets and reference them in init scripts

: Databricks Repos integrates with Git providers (like GitHub, GitLab, and Azure DevOps), enabling version control for notebooks and other project files. This allows for collaboration, branching, and maintaining a history of changes, which are fundamental to CI/CD workflows.

Databricks Repos supports operations like pulling the latest version of code via API and resolving merge conflicts. However, other operations like merging code, handling pull requests, and deleting branches are managed in Git providers.

Correct. Repos allow selective staging of changes and adding comments for commits, which is not supported in basic notebook versioning.

Structured streaming:

Structured streaming ensures exactly-once data processing by designating streaming sinks as idempotent

CDC:

Using CDC with Delta Live Tables (DLT) breaks the append-only requirements for streaming table sources because CDC allows for data updates and deletions in the target table, which goes against the append-only nature of streaming tables.

Compute: